

Chapter 12.

Service mesh



Lee youngkon



A problem situation

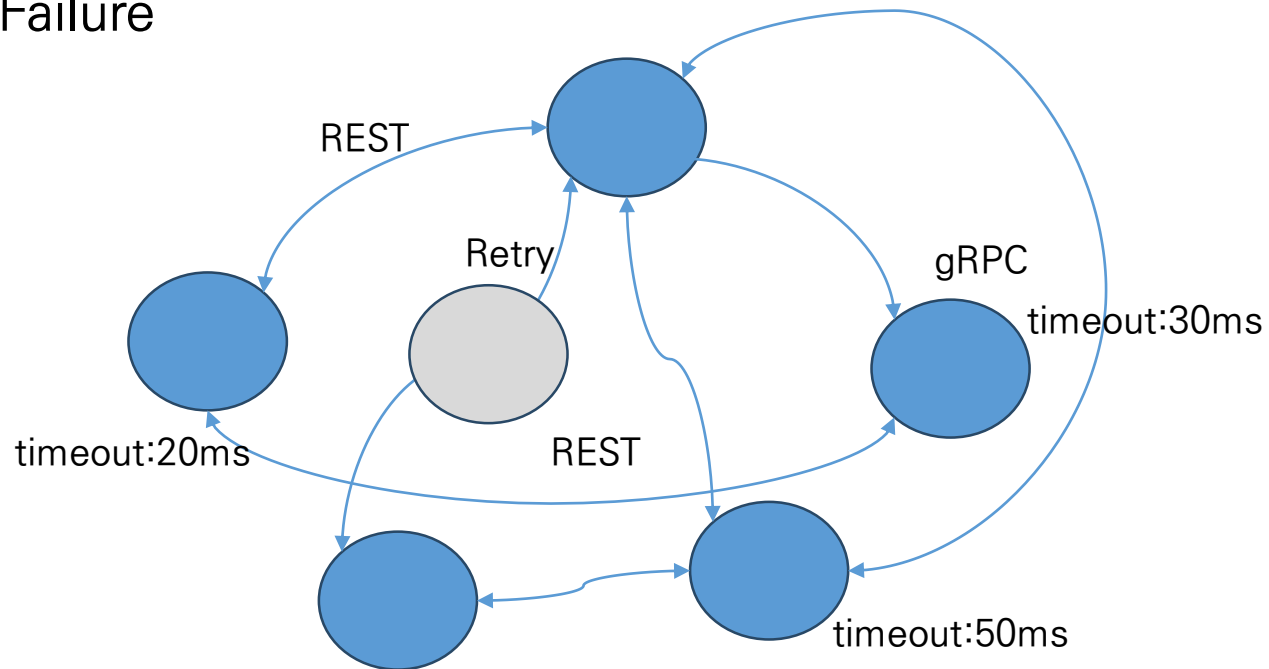
- ✓ Increased microservices → Increased communication complexity
- ✓ Call path between services unclear
- ✓ Difficulty tracking causes in the event of a failure



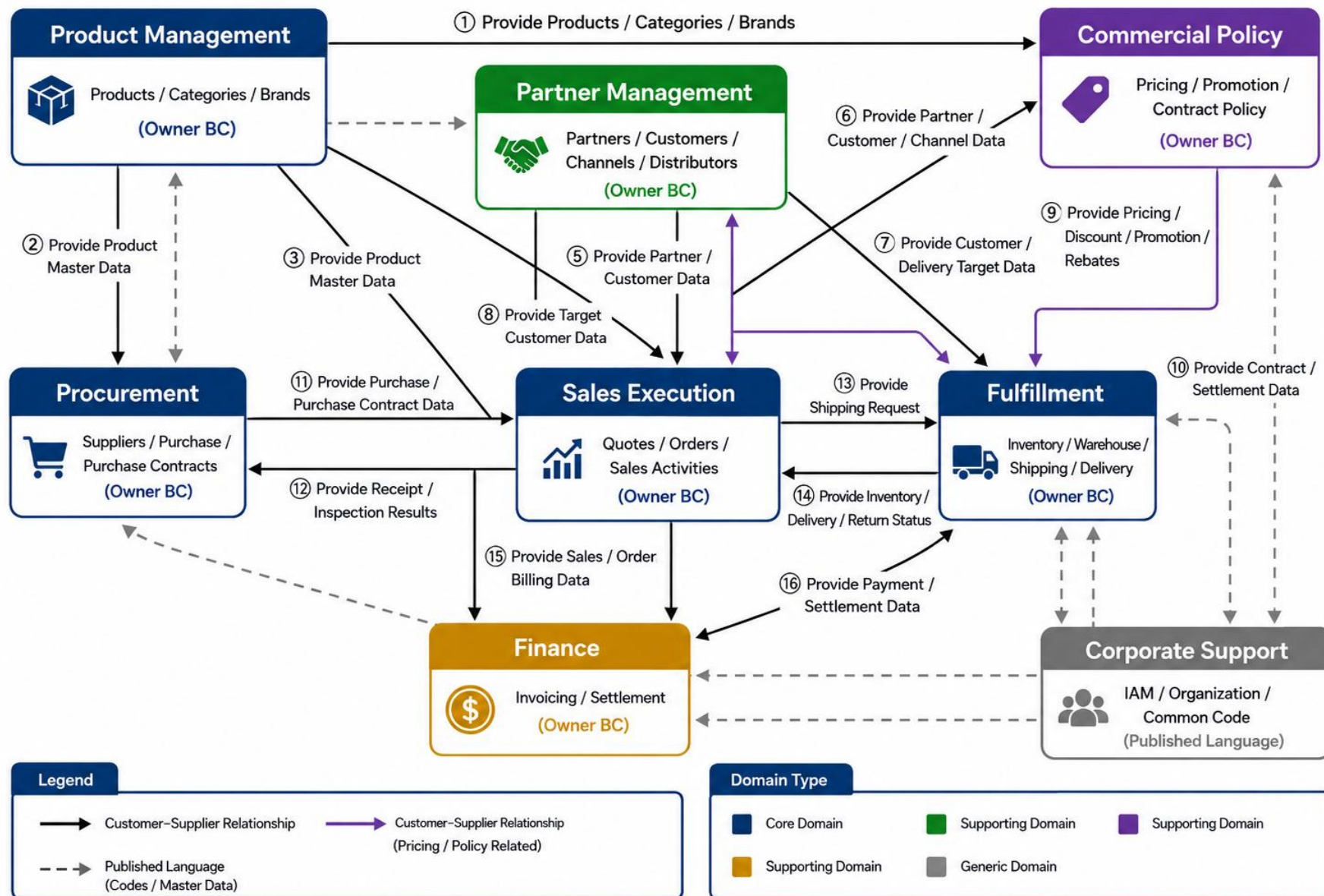
Communication
between services is
key to the system

Communication problems in MSA environments

- ✓ Proliferation of Inter-Service REST Calls
- ✓ Timeout/retry logic redundancy implementation for each service
- ✓ Security (TLS) Direct Implementation Required
- ✓ Cascading Failure



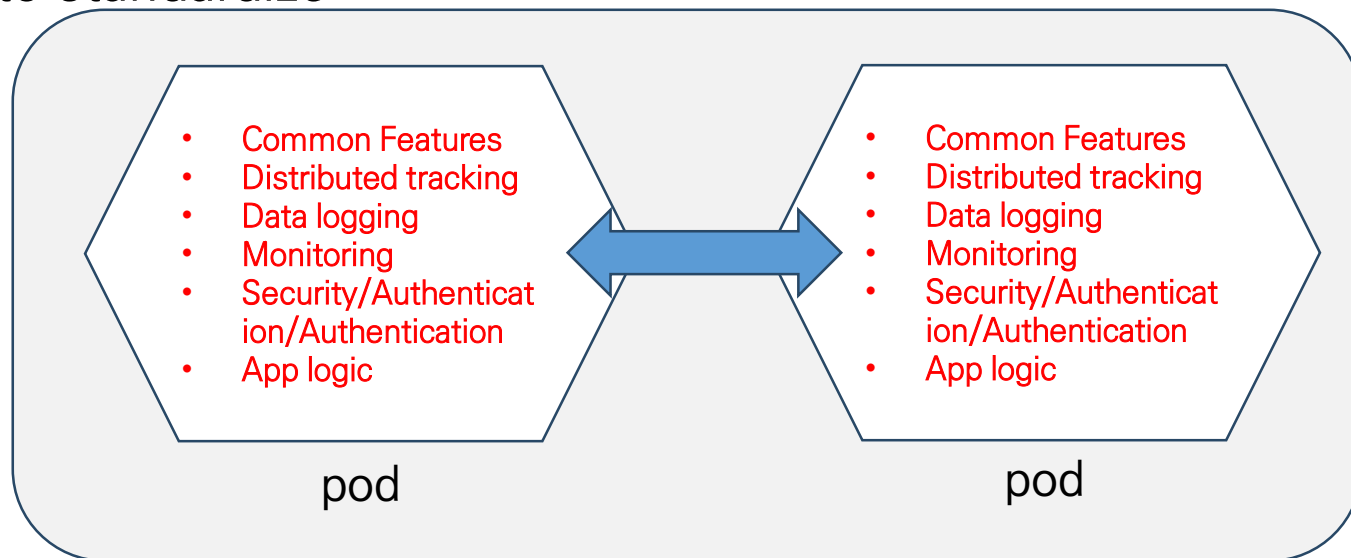
01 Need for service mesh



01 Need for service mesh

Solution plan

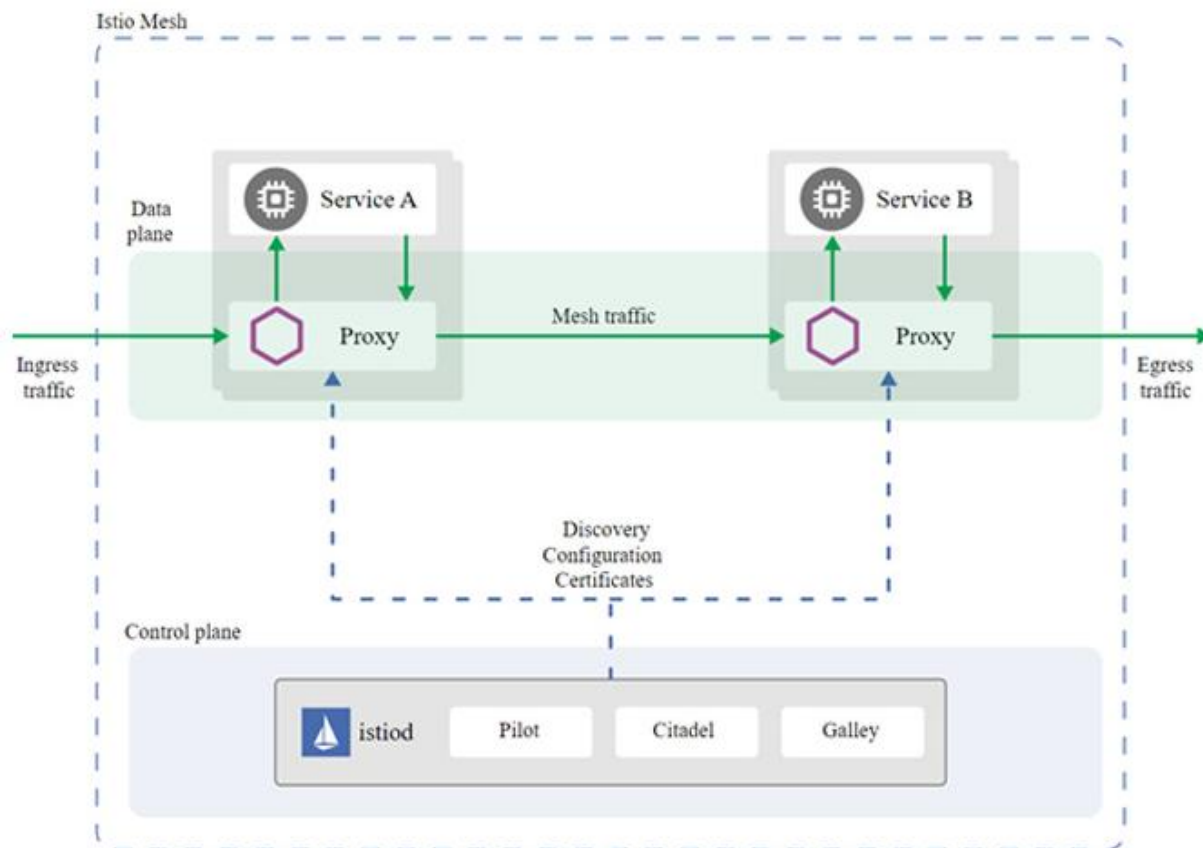
- ✓ Retry, timeout, direct circuit breaker implementation for pod
- ✓ problem
 - Redundant implementation per service
 - Difficulty in maintenance
 - Unable to standardize



01 Need for service mesh

Solution plan

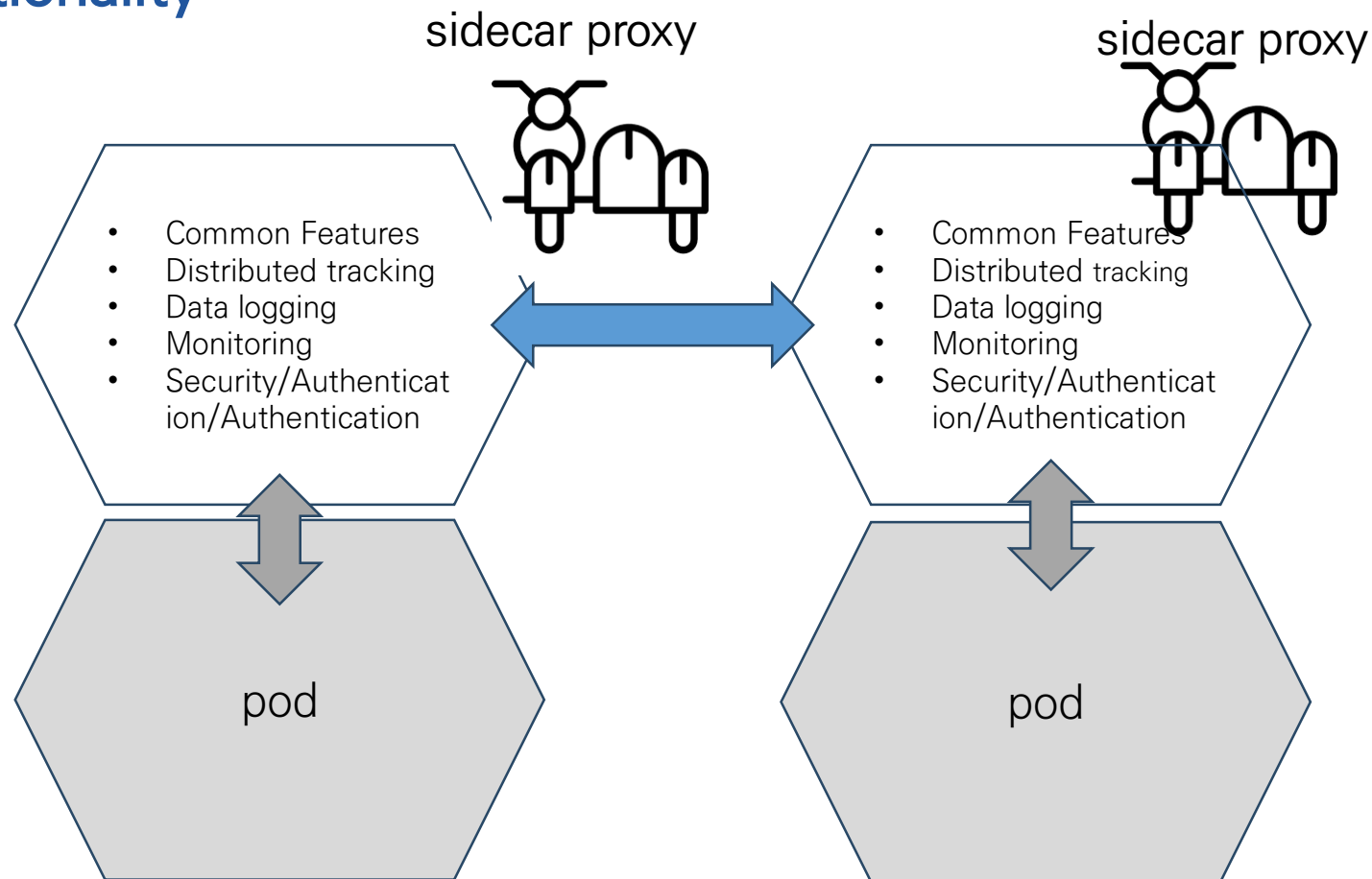
- ✓ Structure to control communication between services at the infrastructure level
- ✓ Characteristics
 - Operate without modifying application code
 - Proxy handles communications



01 Need for service mesh

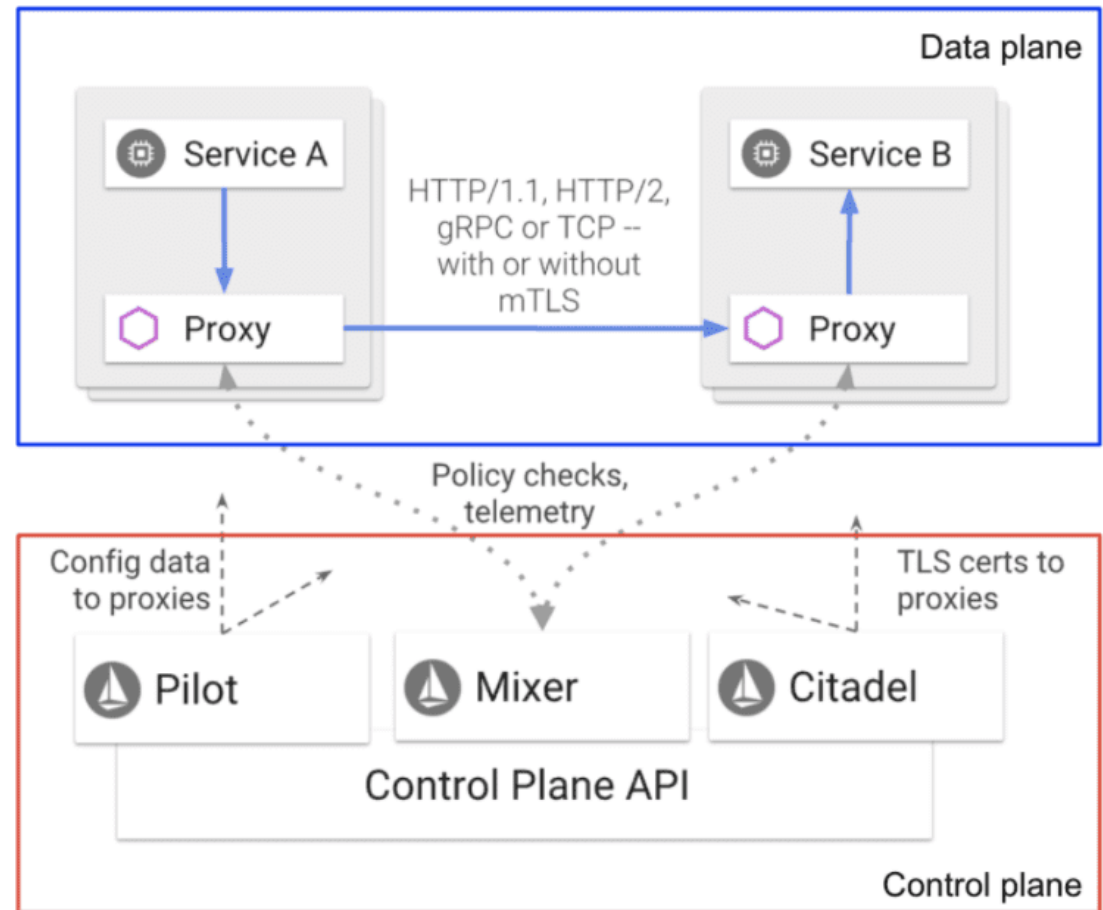
Service mesh functionality

- ✓ Traffic Control
- ✓ Distributed Tracking
- ✓ Data logging
- ✓ Monitoring
- ✓ Security
- ✓ Authentication



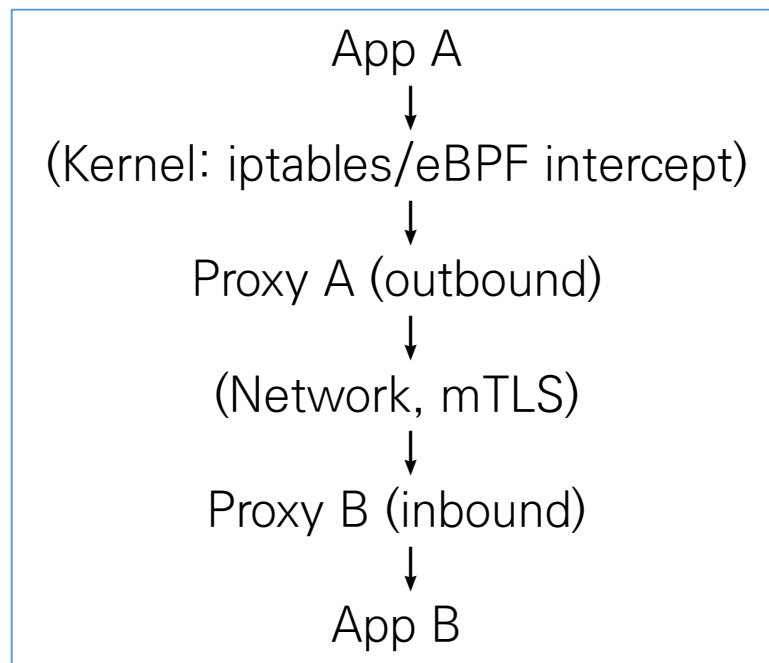
Key Features

- ✓ Traffic Control
 - Load balancing
 - Retry / Timeout
- ✓ Observability
 - Latency
 - Success rate
 - Call graph
- ✓ Security
 - Automatically apply mTLS



proxy communication structure

- ① Service A calls a generic service name
- ② iptables forces traffic to Envoy in A
- ③ A Envoy receives B endpoint information from Istiod
- ④ istiod delivered via xDS API:
 - B Service Pod List
 - Envoy address of each Pod
 - Load Balancing Policy
 - mTLS certificate
 - Routing Rules
- ⑤ A Envoy selects the appropriate B Envoy
 - e.g :Round Robin, Least Request, Canary
 - Routing, Version Routing



Weakness

- ✓ Resource overhead (most realistic issue)
 - Add sidecar proxy to all pods
 - Increasing CPU/Memory (approximately 10% to 30%)
 - Need to increase the number of nodes
- ✓ Increased network latency
 - Every request goes through the proxy
 - App → Proxy → Network → Proxy → App
 - Increased latency (several ms to several tens of ms)

Weakness

- ✓ Increasing Debugging Difficulty
 - When a problem occurs: location?– Application?– Network?– Proxy?– Mesh policy?
 - Difficulty in separating causes
- ✓ Increasing architectural complexity
 - Additional elements: Control Plane, Data Plane (proxy), CRD (ServiceProfile, etc.), Policy (retry, timeout, etc.)
- ✓ the cost of learning
 - mTLS Concepts
 - Traffic Policy
 - observability
 - Sidecar structure

Introduction Considerations

- ✓ Is it really necessary
 - Number of services (10 or more)
 - Inter-Service Call Complexity
 - Disability Analysis Difficulties
 - Observability required
- ✓ Retry / timeout policy design
 - Too aggressive → Failure amplification
 - Too conservative → Failure detection delay
- ✓ Consideration of mTLS impact
 - Internal communication encryption → CPU usage increases
 - Change the authentication scheme

Introduction Considerations

- ✓ Preparation for utilization of observability: 70% of reasons for introducing mesh are due to observability
 - p95 latency
 - success rate
 - call graph
- ✓ Operational automation required
 - Monitoring (Prometheus, Grafana)
 - Alarm
 - Collect Logs

Installation Order

- ✓ Install WSL or Gitbash (gitbash is the simplest)
- ✓ Install Gateway API
- ✓ Install Linked CRD
- ✓ When install Linkerd control plane `--set proxyInit.runAsRoot=true` add
- ✓ Verified with linkerd check

Linkerd installation script (based on docker desktop)

- ✓ `linkerd uninstall | kubectl delete -f -`
 - ✓ `kubectl delete ns linkerd`
 - ✓ `curl -sL https://run.linkerd.io/install | sh`
 - ✓ `kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/standard-install.yaml`
 - ✓ `linkerd install --crds | kubectl apply -f -`
 - ✓ `linkerd install --set proxyInit.runAsRoot=true | kubectl apply -f -`
 - ✓ `linkerd check`
 - ✓ `kubectl get pods -n linkerd`
 - ✓ `kubectl get configmap -n linkerd`
- You don't need to install for the first time
- Reflect path after installing Linkerd CLI

Sidecar injection target

- ✓ REST API (pub/sub, producer/consumer etc.)
- ✓ gRPC
- ✓ Internal TCP Calls
- ✓ mTLS
- ✓ Traffic Control

Subject to careful consideration when injecting sidecar

- ✓ Event Brokers/MessageQueue Like Kafka, RabbitMQ
- ✓ It has its own security features, clustering, and port structure

Check after injecting Linkerd

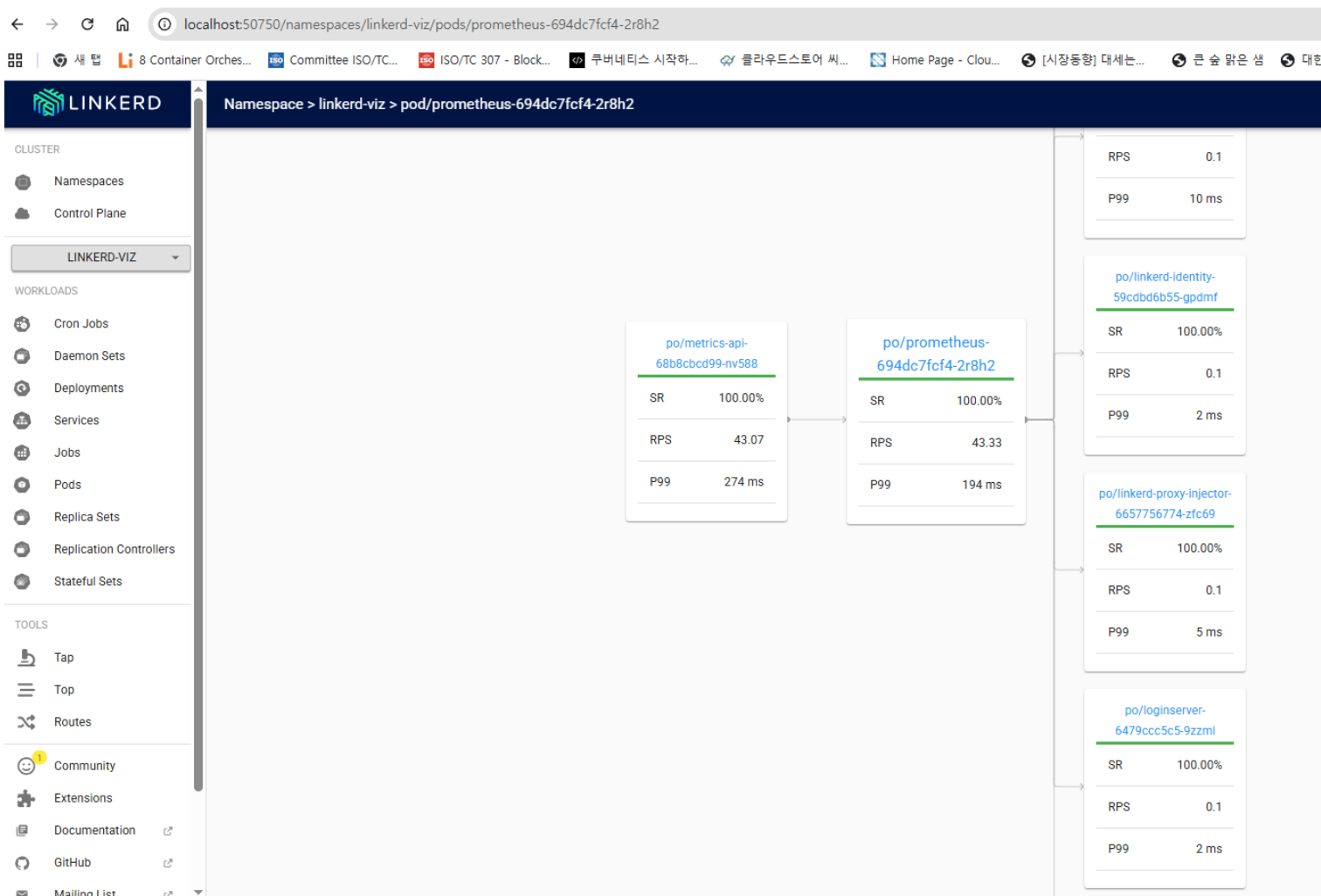
- ✓ There are 2 containers in the pod
 - `kubectl get pods`
 - `kubectl describe pod <pod name>`
- ✓ app container + linkerd-proxy
- ✓ All inter-service calls via proxy (OK)
 - `linkerd viz install | kubectl apply -f -`
 - `linkerd viz check`
 - `linkerd viz stat deploy -A` (statistics)
 - `linkerd vizedges deploy -A` (messaging information)
 - `linkerd viz dashboard` (dashboard screen)

LinkedBiz Dashboard: Namespace

The screenshot shows the Linkerd dashboard interface. The left sidebar contains a navigation menu with 'Namespaces' selected under the 'CLUSTER' section. The main content area displays 'HTTP Metrics' for various namespaces. The 'default' namespace is highlighted with a red circle. The table below shows the metrics for each namespace.

Namespace ↑	↑ Meshed	↑ Success Rate	↑ RPS	↑ P50 Latency	↑ P
book-dev	0/2	---	---	---	
default	6/6	100.00% ●	1.98	1 ms	
ingress-nginx	0/1	---	---	---	
kube-node-lease	0/0	---	---	---	
kube-public	0/0	---	---	---	
kube-system	0/9	---	---	---	
linkerd	3/3	100.00% ●	2.08	1 ms	
linkerd-viz	5/5	100.00% ●	3.58	2 ms	

Check Linkerd metric information (openfeignclient and resttemplateserver)



Meaning of terminology

- ✓ SR (Success Rate)
 - success rate
- ✓ RPS (Requests Per Second)
 - Requests per second
 - 예: prometheus: 45.1 req/sec, mysql: 0.1 req/sec
- ✓ P99
 - 99% of requests are processed less than this time
 - mysql: 3ms, prometheus: 99ms

Linkerd yaml Description

```
apiVersion: linkerd.io/v1alpha2
kind: ServiceProfile
metadata:
  name: payment.default.svc.cluster.local
  namespace: default
spec:
  routes:
    - name: GET /payment
      condition:
        method: GET
        pathRegex: /payment
        isRetryable: true
        timeout: 1s
      retryBudget:
        retryRatio: 0.2
        minRetriesPerSecond: 10
        ttl: 10s
```

Retry Rate Limit.
Up to 2 out of 10
requests
allowed for retry

Minimum Retry
Count per Second

Will you base your
requests on recent
seconds?

Linkerd yaml

- ✓ Time out
 - timeout: Failure handling when exceeds timeout
 - avoid response delays, mitigate system-wide bottlenecks
 - enforce request failure processing, upstream connection termination, error return to client. Retry if Retry Condition
- ✓ retryRatio: 0.2  Allow retries up to 20% of the original request volume.
 - Example: 100 requests per second → up to 20 retries
- ✓ minRetriesPerSecond: 10  Specify at least 10 retries per 1 second
 - Devices for ensuring retries
 - For example, if retryRatio is 0.2, almost no retry occurs
- ✓ Time to Live (TTL): 10 => Based on requests for the last 10 seconds

Delete mesh (run on gitbash)

- ✓ `linkerd viz uninstall | kubectl delete -f -`
- ✓ `linkerd uninstall | kubectl delete -f -`
- ✓ `kubectl delete crd -I linkerd.io/control-plane-ns=linkerd`
- ✓ `kubectl delete namespace linkerd`
- ✓ `kubectl delete namespace linkerd-viz`
- ✓ `kubectl rollout restart deployment`

Installing the Istio

- ✓ `Invoke-WebRequest -Uri https://github.com/istio/istio/releases/download/1.27.3/istio-1.27.3-win.zip -OutFile istio.zip`
- ✓ `Expand-Archive istio.zip -DestinationPath .`
- ✓ `cd istio-1.27.3`
- ✓ `istioctl install --set profile=demo -y`
- ✓ `kubectl get pods -n istio-system`
- ✓ `kubectl label namespace default istio-injection=enabled`
- ✓ `kubectl apply -f samples/addons`
- ✓ `kubectl get ns --show-labels`
- ✓ `kubectl get pods -n istio-system`
- ✓ `istioctl dashboard kiali`
- ✓ `istioctl dashboard grafana`
- ✓ `istioctl dashboard prometheus`
- ✓ `kubectl rollout restart deployment`

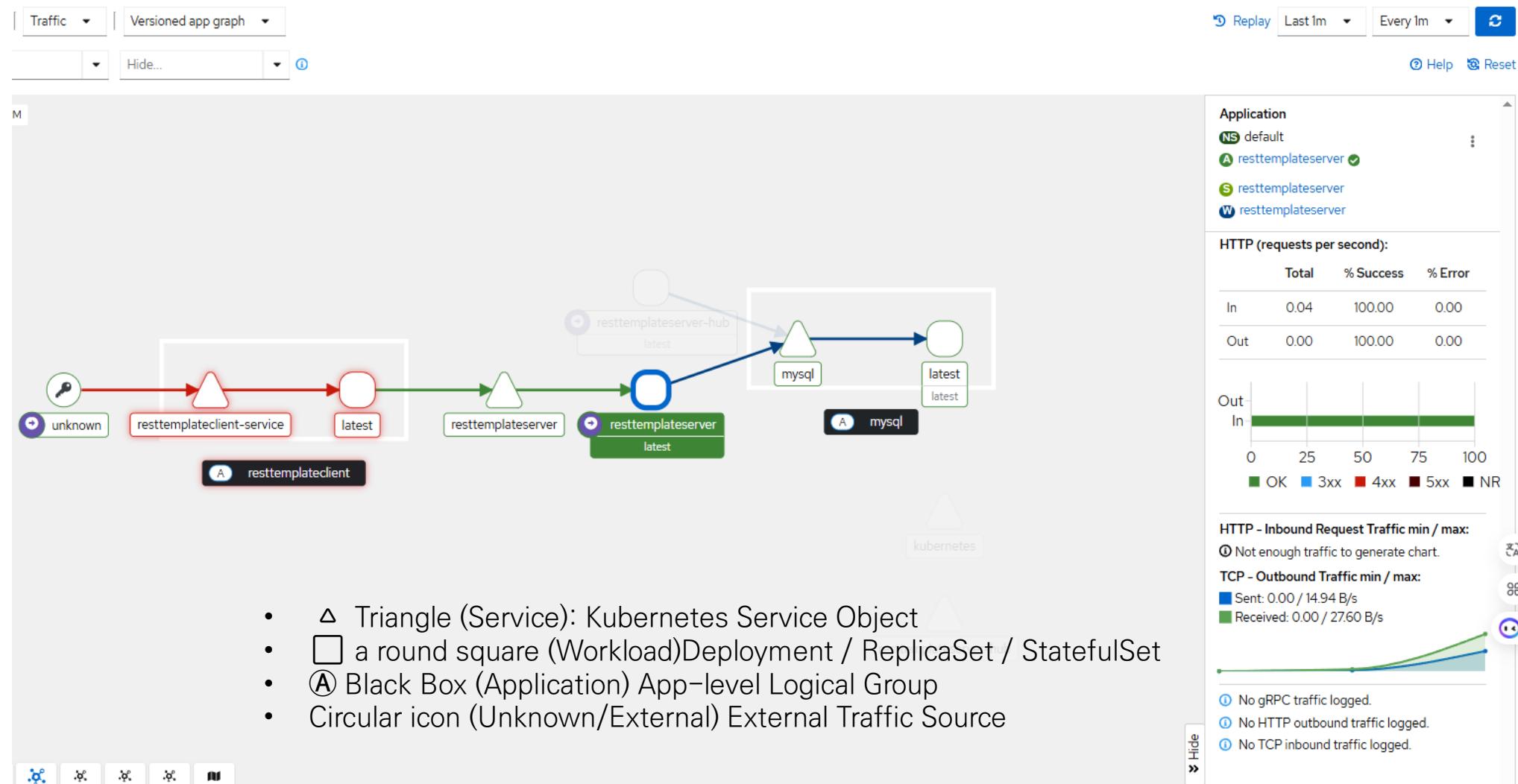
* Run in powershell

When delete: `istioctl uninstall --purge`

Kiali

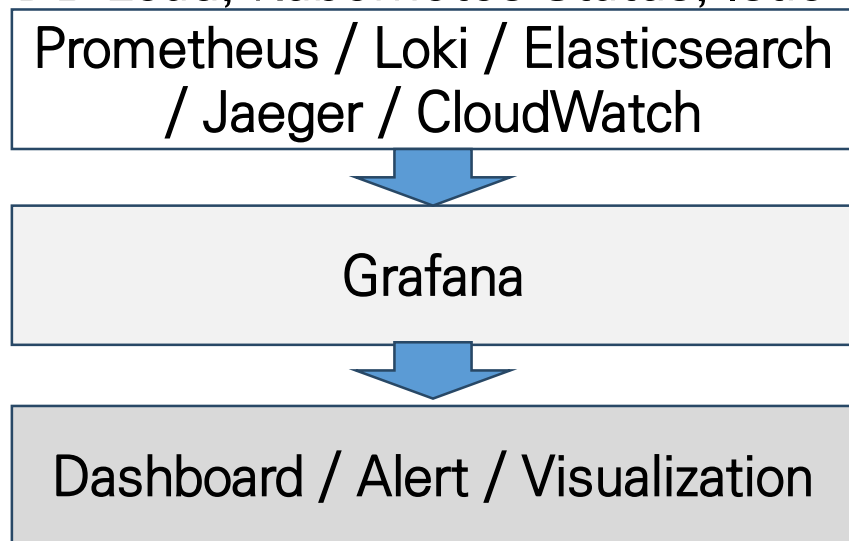
- ✓ Observability console to visually manage and analyze service mesh environments
 - A management dashboard that shows call relationships between services like a real-time map
- ✓ Visualize Service Mesh Topology
- ✓ Check Traffic Flow
 - Request Rate, Success Rate, Error Rate, Response Time
- ✓ Check the status of the side car
 - Siderar presence, Missing Siderar detection, mTLS application
- ✓ Verifying Settings
 - VirtualService. DestinationRule, Gateway, AuthorizationPolicy
- ✓ Security Status
 - mTLS enabled, encrypted, and inter-service security policy

Kiali



Grafana

- ✓ An integrated monitoring dashboard platform that visualizes metrics, logs, and traces of systems, applications, and infrastructure
 - An analysis tool that displays operational data (performance/status) in real-time charts and dashboards
- ✓ Visualize operational status:
 - CPU, Memory, Pod Status, API Response Speed, Error Rate, Kafka Throughput, DB Load, Kubernetes Status, Istio Service Metrics



Grafana Key Features

- ✓ Dashboard
 - Real-time charts, heatmaps, gauges, tables, alarm panels
- ✓ Alerting
 - CPU over 90%, Error surge, Pod down, Latency increase
- ✓ Log Analysis
 - When Loki is linked: Search for error logs
- ✓ Kubernetes Monitoring
 - Pod status, resources by Namespace, Deployment performance
- ✓ When used with Istio
 - Request Rate, Success Rate, P95/P99 Latency, Service Traffic, mTLS part of the condition

Grafana Key Features (Check Logs)

Kubernetes + Grafana + Loki + Promtail Log Builds

- ✓ Pod Logs
 - Collect Promtail Logs
 - Save Loki Logs
 - Look up Grafana
- ✓ `kubectl get svc -n istio-system | findstr loki` (Check Lockie Installation)

Grafana Key Features (Check Logs: Install promtail)

- ✓ `winget install Helm.Helm` (powershell) New window after installation
- ✓ `helm repo add grafana https://grafana.github.io/helm-charts`
- ✓ `helm repo update`
- ✓ `kubectl create namespace monitoring`
- ✓ `kubectl port-forward -n istio-system svc/loki 3100:3100`
- ✓ `helm install promtail grafana/promtail --namespace monitoring --set config.clients[0].url=http://loki.istio-system.svc.cluster.local:3100/loki/api/v1/push`
- ✓ `kubectl get pods -n monitoring`
- ✓ `kubectl logs -n monitoring daemonset/promtail`

Grafana Key Features (check log load with promtail => Loki)

- ✓ Invoke-WebRequest <http://localhost:3100/loki/api/v1/labels>
 - {"status":"success","data":["namespace","pod","container","app",...]}
- ✓ Register Loki Data Source in Grafana
 - <http://loki.istio-system.svc.cluster.local:3100>
- ✓ Grafana Log Inquiry (using LogQL) (Run in Code, not Builder)
 - {namespace="default"}
 - {app="resttemplateserver"}
 - {namespace="default"} |= "ERROR"

03 Utilize Service Mesh

Check the Grafana log

The screenshot displays the Grafana web interface in dark mode. The left sidebar contains navigation links: Home, Starred, Dashboards, Explore (highlighted with a red circle), Drilldown, Metrics, Logs, Traces, Profiles, Alerting, Connections (highlighted with a red circle), Add new connection, Data sources, and Administration. The main panel shows the 'Explore' view for the 'Loki' data source. The query editor contains the query `{app="resttemplateclient"}` (highlighted with a red circle). Below the query, a single log entry is shown: `1 {app="resttemplateclient"}`. The 'Options' section indicates 'Type: Range', 'Line limit: 1000', and 'Direction: Backward'. At the bottom, a 'Logs volume' bar chart shows log volume over time, with a red bar indicating an 'error' at 19:25. The 'Logs' tab is selected at the bottom, showing 4 lines displayed and total bytes processed: 1.74 kB. Common labels are listed: `app=resttemplateclient container=istio-proxy job=default/resttemplateclient` (highlighted with a red circle).

Grafana Key Features (Register Prometheus as Data Source)

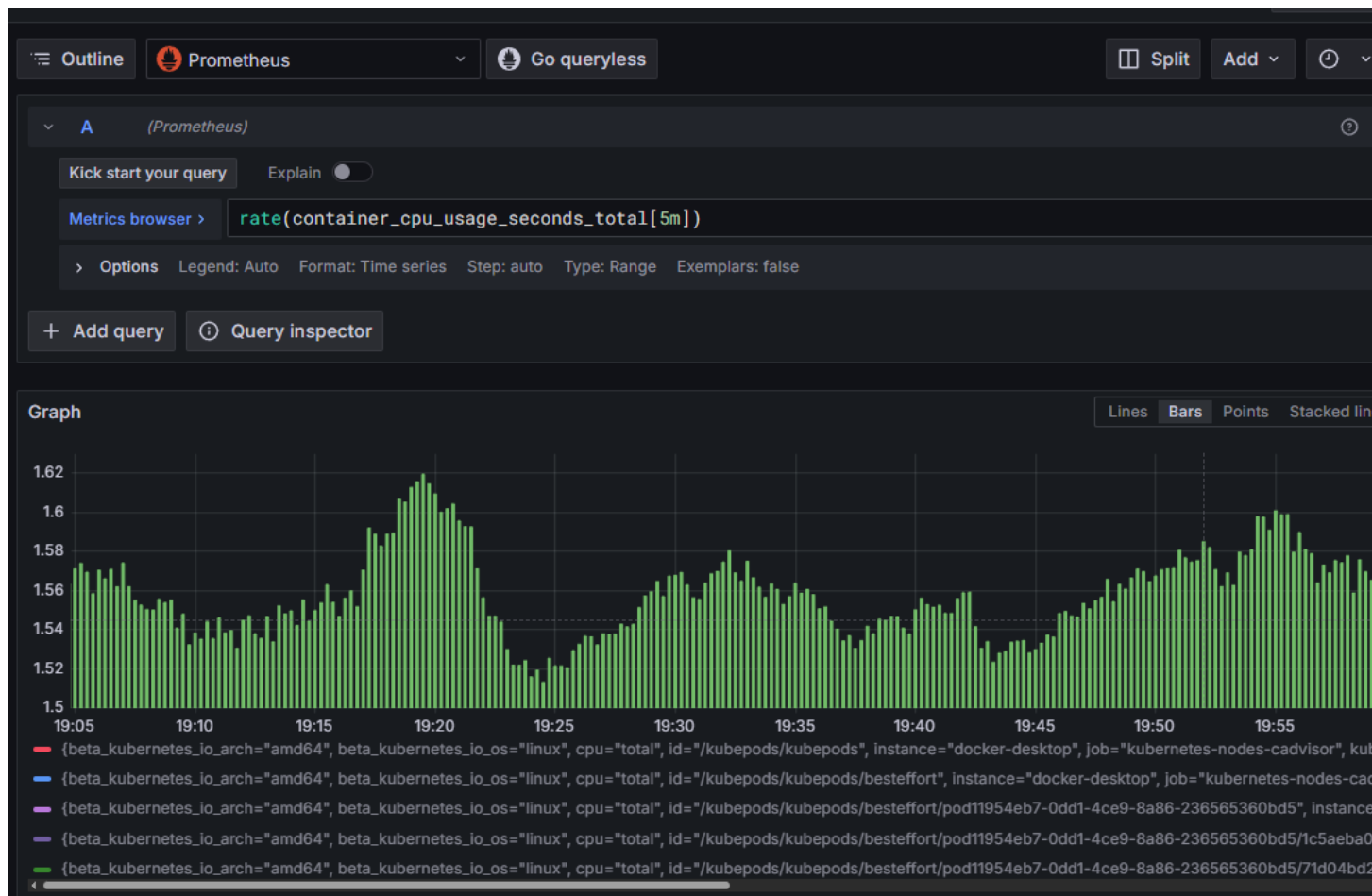
- ✓ Register Data Source
 - <http://prometheus.istio-system.svc.cluster.local:9090>

category	Prometheus UI	Grafana
PromQL test	very good	Good
Dashboard	restrictive	strong
Visualization	basics	high quality
Log Inquiry	impossibility	Loki interlocking

03 Utilize Service Mesh

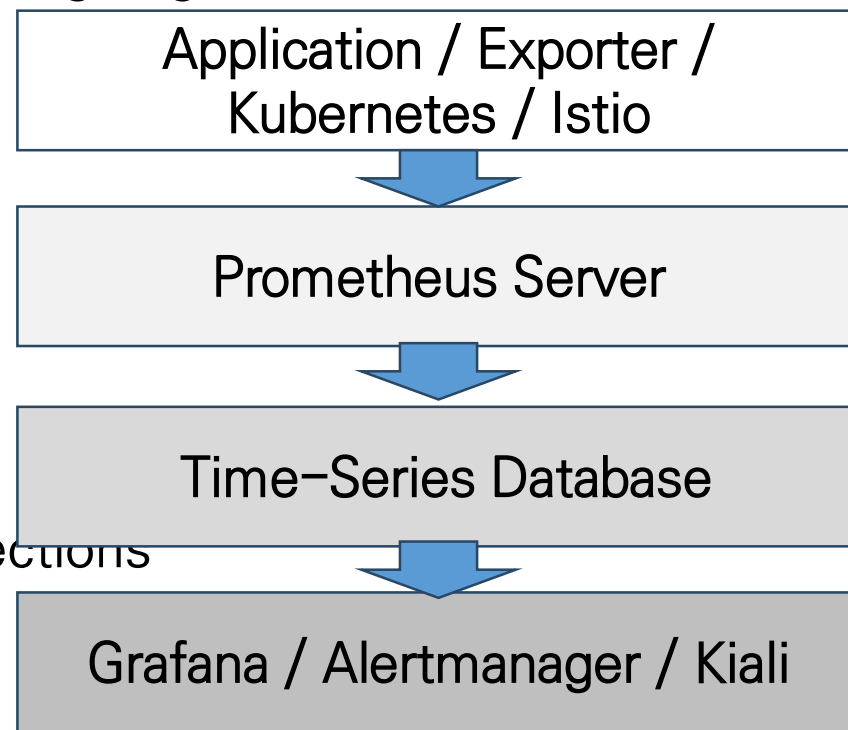
Grafana Key Features (Register Prometheus as Data Source)

PromQL test: `rate(container_cpu_usage_seconds_total[5m])`



Prometheus Concept

- ✓ An open-source monitoring system that collects, stores, and queries metrics for systems and applications
- ✓ Get and save the app's health figures periodically
 - Time series database + monitoring engine
- ✓ Key roles (periodic collection)
 - CPU utilization
 - Memory Usage
 - Number of HTTP requests
 - Error rate, response time
 - Kafka lag, number of DB connections



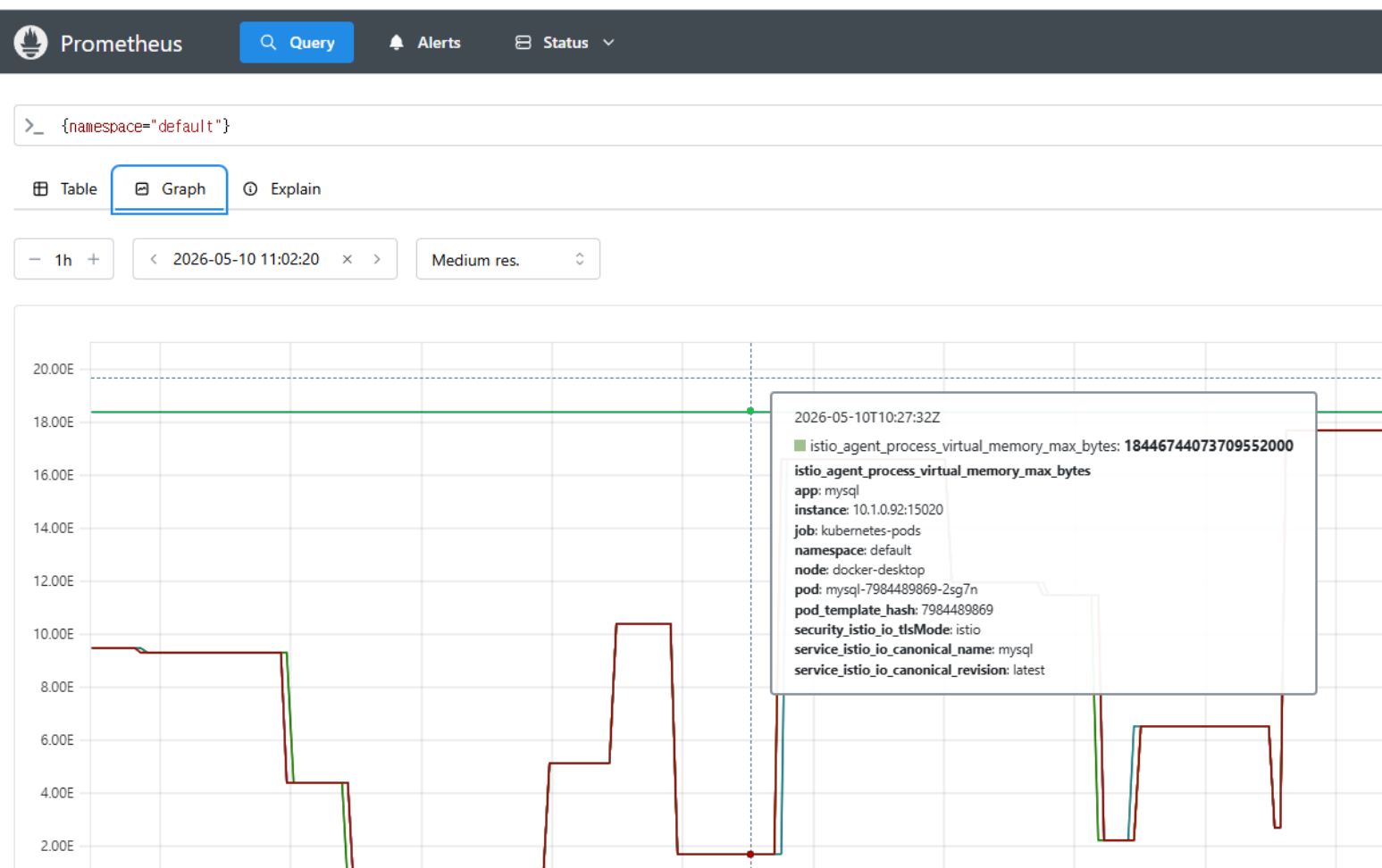
Prometheus Concept

- ✓ Exporters (metric providers)
 - node export, MySQL export, kafka export, cAdvisor, kube-state-metrics
- ✓ Using the Pull Method
 - Prometheus requests the target directly: GET/Metrics
- ✓ Time series DB
 - Time-based storage: `cpu_usage{pod="api-1"}75`
- ✓ PromQL (dedicated query language): `Speed (http_requests_total[5m])`
- ✓ Monitorable: Nodepot, Deployment, Namespace, HPA, Service Mesh, API Latency
- ✓ Istio environment: number of requests, error rate, service latency, mTLS telemetry
- ✓ Link notification managers: CPU > 90%, Ford crash loop, 5xx surge

Utilize Service Mesh

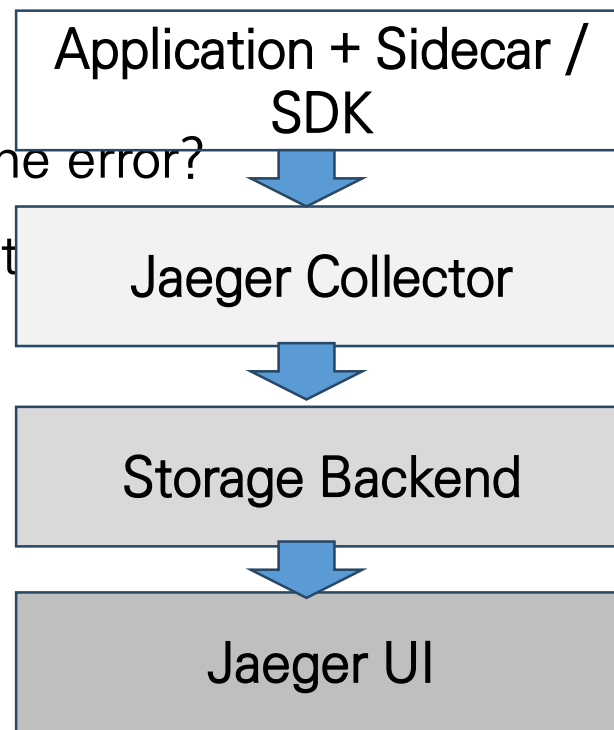
Run Prometheus

`sum(rate(container_cpu_usage_seconds_total{pod=~"restt
emplateserver.*", container!="POD"}[5m])) by (pod)`



Jaeger Concept

- ✓ Systems that track the flow of requests between microservices
 - A tool that tracks over time how a single user request is processed across multiple services
 - Automatically create a trace with Istio sidecar
 - Record each service call in Trace/Span
 - Where did it slow down? Which service is the error?
 - What's the bottleneck? What's the order of the steps?
- ✓ a core concept
 - Trace: One complete request flow
 - Span: Each Service Step



Jaeger Features

- ✓ Visualize the Call Path
 - Service order, parallel call, delay interval
- ✓ Analysis of Latency
 - Slow service, DB query delay
- ✓ Error tracking
 - Failed service, timeout, retry
- ✓ Root Cause Analysis
 - e.g :payment-service DB timeout

tracing.yaml

```
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  meshConfig:
    enableTracing: true
    defaultConfig:
      tracing: {}
    extensionProviders:
      - name: jaeger
        opentelemetry:
          port: 4317
          service: jaeger-collector.istio-
system.svc.cluster.local
```

```
istioctl install -f tracing.yaml -y
```

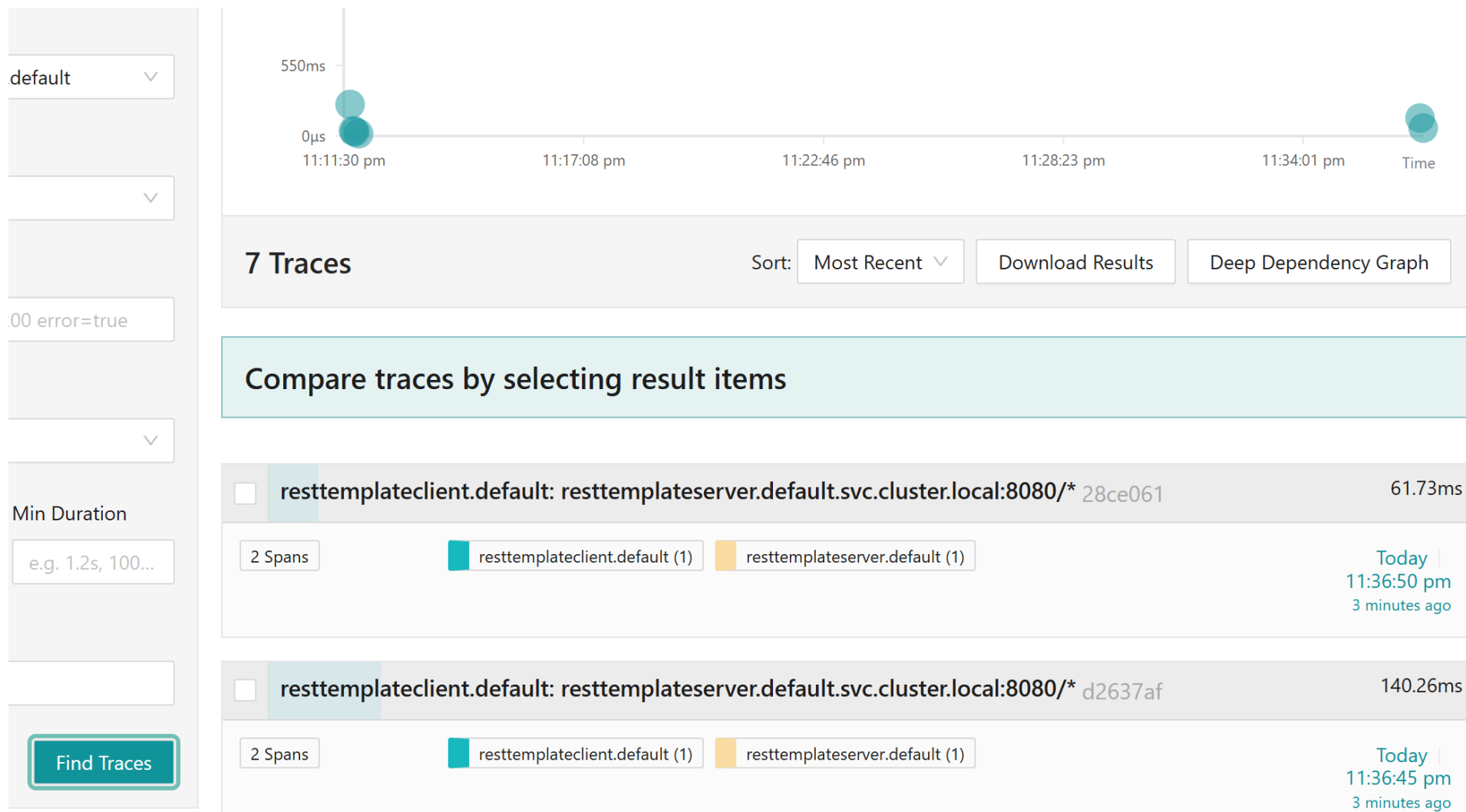
telemetry.yaml

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
  namespace: istio-system
spec:
  tracing:
    - providers:
        - name: jaeger
      randomSamplingPercentage: 100
```

```
kubectl apply -f telemetry.yaml
```

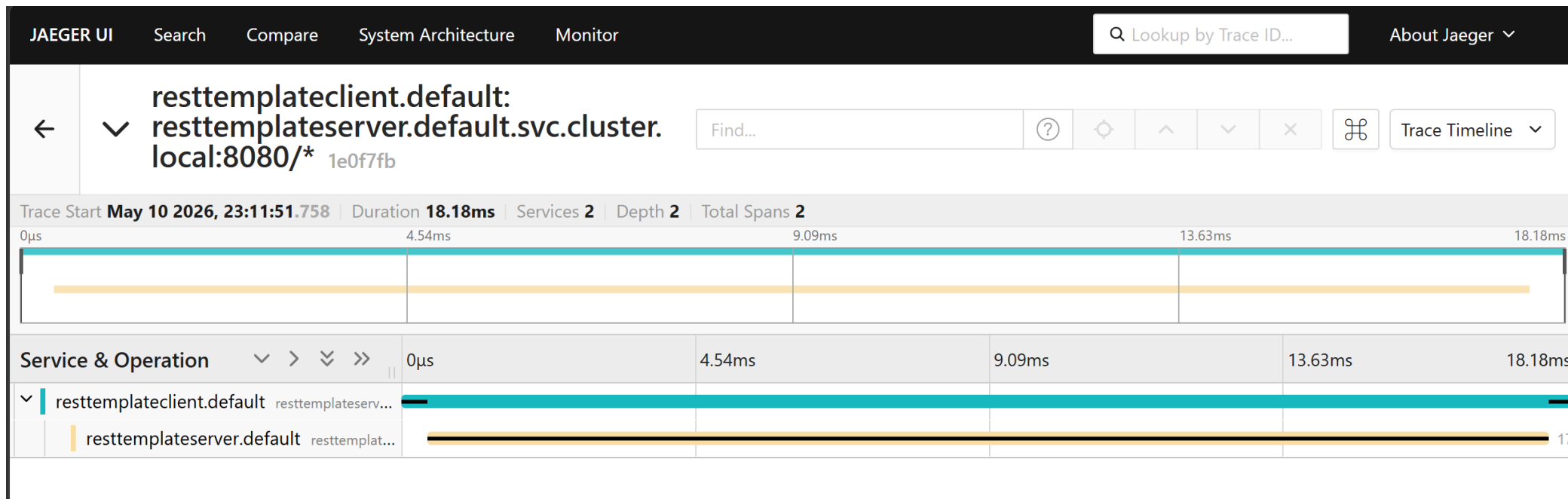
03 Utilize Service Mesh

Jaeger Run Results



03 Utilize Service Mesh

Jaeger Run Results



- If it doesn't show up, make sure it's on the sidecar: `kubectl get pod restemplateserver-padname -ojsonpath="{.spec.containers[*].name}"`
- If not, sidecar injection: `kubectl annotation distribution restorerver sidecar.istio.io/inject = "true" --overwrite`
- `kubectl label namespaces default istio.io/rev=default --overwrite`
- Redistribution: `kubectrolout restarts deployment`

Service Mesh G/W

✓ G/W Default Policy Settings:

- Open Port
- domain
- HTTP/HTTPS
- TLS

```
kubectl port-forward -n istio-system  
svc/istio-ingressgateway 8080:80
```

```
apiVersion: networking.istio.io/v1beta1  
kind: Gateway  
metadata:  
  name: resttemplate-gateway  
  namespace: default  
spec:  
  selector:  
    istio: ingressgateway  
  servers:  
    - port:  
        number: 80  
        name: http  
        protocol: HTTP  
      hosts:  
        - "*"
```

Virtual Service

- ✓ Virtual Service role:
- Path Routing
 - Version Routing
 - Header-based routing decisions
 - Retry / Timeout / Fault Injection



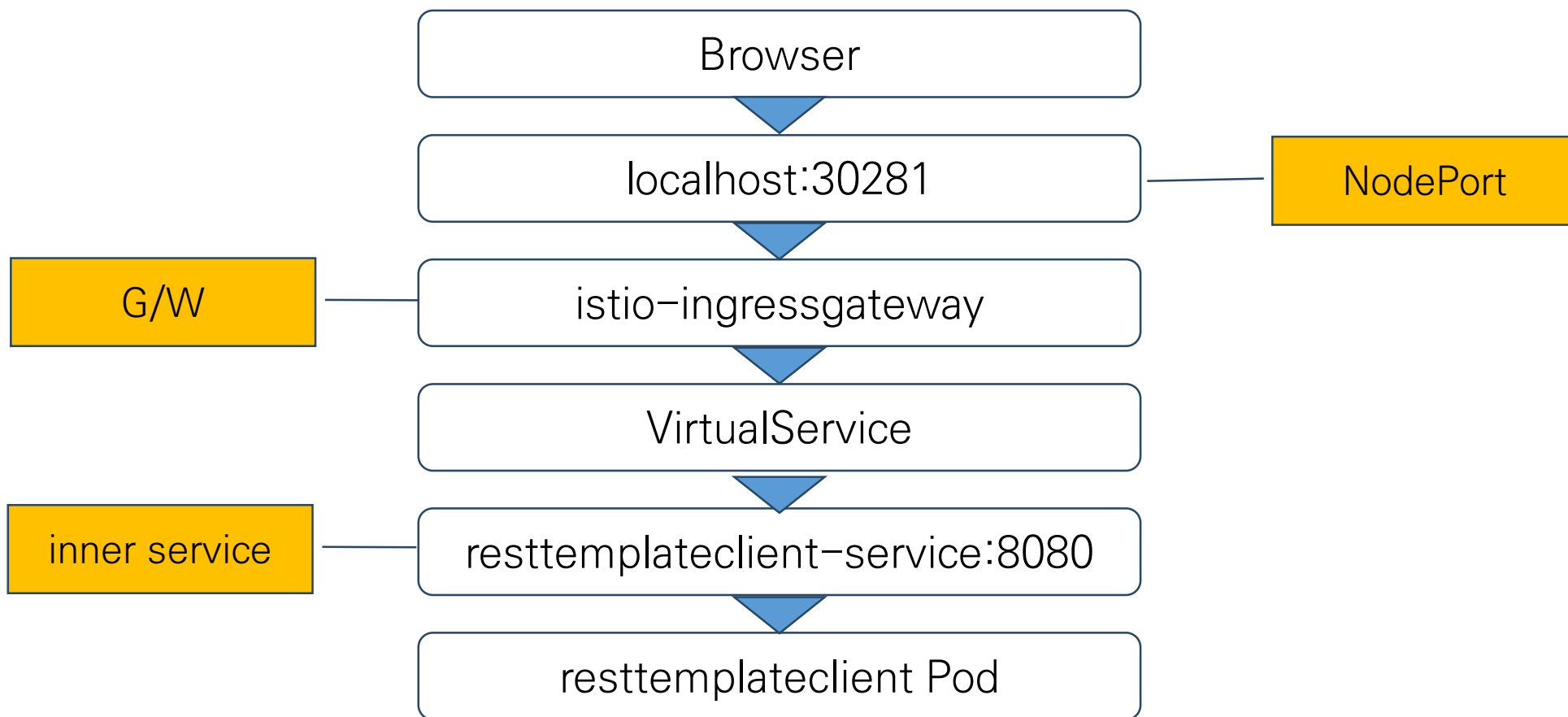
Decide where
to send it

<http://localhost:8080/books>

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: resttemplateclient-vs
  namespace: default
spec:
  hosts:
    - "*"
  gateways:
    - resttemplate-gateway
  http:
    - match:
        - uri:
            prefix: /books
      route:
        - destination:
            host: resttemplateclient-
              service.default.svc.cluster.local
            port:
              number: 8080
```

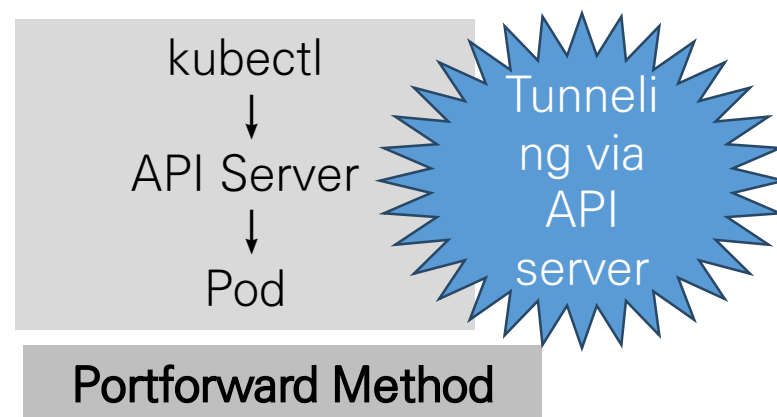
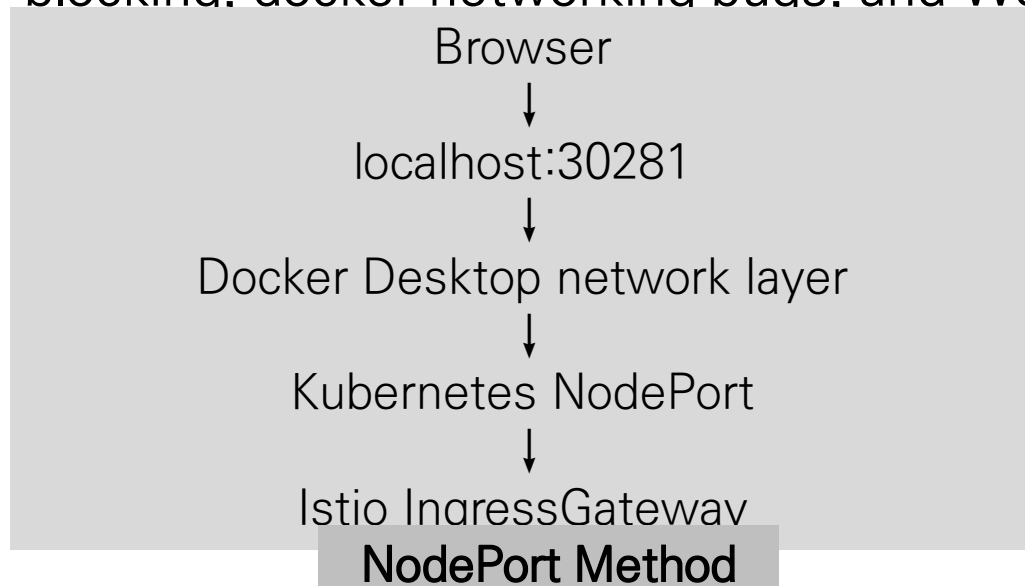
G/W + Virtual Service

- ✓ Go to the web browser: <http://localhost:30281>



G/W + Virtual Service

- ✓ `http://localhost:30281` → NodePort method not connected
- ✓ Proceed to port forwarding: `kubectl port-forward -n istio-system svc/istio-ingressgateway 8088:80`
- ✓ `http://localhost:8088/index.html`, `http://localhost:8088/books`
- ✓ Docker desktops are not physical nodes. This can cause a variety of problems, including port collisions, local host redirection failures, firewall blocking, docker networking bugs, and WSL2 bridge issues



If you need to use NodePort/LoadBalancer: Production point

- ✓ Use at production point
 - Actual External Services
 - Public IP
 - Cloud LB
 - MetalLB

Additional settings in Virtual Service

- ✓ timeout
- ✓ retries
- ✓ retryOn
- ✓ fault injection
- ✓ route weight
- ✓ header routing

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: resttemplateclient-vs
spec:
  hosts:
    - "*"
  gateways:
    - resttemplate-gateway
  http:
    - match:
        - uri:
            prefix: /books
      timeout: 3s
      retries:
        attempts: 3
        perTryTimeout: 1s
        retryOn: gateway-error,connect-failure,refused-stream
      route:
        - destination:
            host: resttemplateserver.default.svc.cluster.local
            port:
              number: 8080
```

Fault Injection

- ✓ Intentionally for some requests:
 - (delay)
 - Occurring an abort
- ✓ Ability to test the resilience of the system

```
fault:  
  delay:  
    percentage:  
      value: 50  
    fixedDelay: 2s
```

2 seconds delay in
50% request

```
fault:  
  abort:  
    percentage:  
      value: 20  
    httpStatus: 500
```

500 error on 20%
request

Deploy Canary

- ✓ How to deploy a new version (v2) progressively exposes it to only a few users before a full deployment

```
route:  
  - destination:  
    host: resttemplateserver  
    subset: v1  
    weight: 90  
  - destination:  
    host: resttemplateserver  
    subset: v2  
    weight: 10
```

The old version is 90% and
the new version is 10%
Increase the new version
by % over time

DestinationRule (Service Stability Policy))

Concept

Policies that define how to handle traffic for destination services

- ✓ loadBalancer
- ✓ connectionPool
- ✓ outlierDetection
- ✓ TLS
- ✓ subsets

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: resttemplateserver-dr
spec:
  host:
    resttemplateserver.default.svc.cluster.local
  trafficPolicy:
    loadBalancer:
      simple: ROUND_ROBIN
    connectionPool:
      tcp:
        maxConnections: 100
      http:
        http1MaxPendingRequests: 50
        maxRequestsPerConnection: 10
    outlierDetection:
      consecutive5xxErrors: 3
      interval: 5s
      baseEjectionTime: 30s
```

Key Settings

- ✓ http1MaxPendingRequests: 50
 - Maximum number of HTTP requests that can be stacked in a queue when the destination service connection pool is full
 - Allow up to 50 queues of current processable connection excess
 - Circuit breaker can be operated when exceeded
- ✓ maxRequestsPerConnection: 10
 - How many HTTP requests can be processed on a single TCP connection before the connection is regenerated
 - Prevent long-term connectivity issues
 - Prevention of specific connection bias

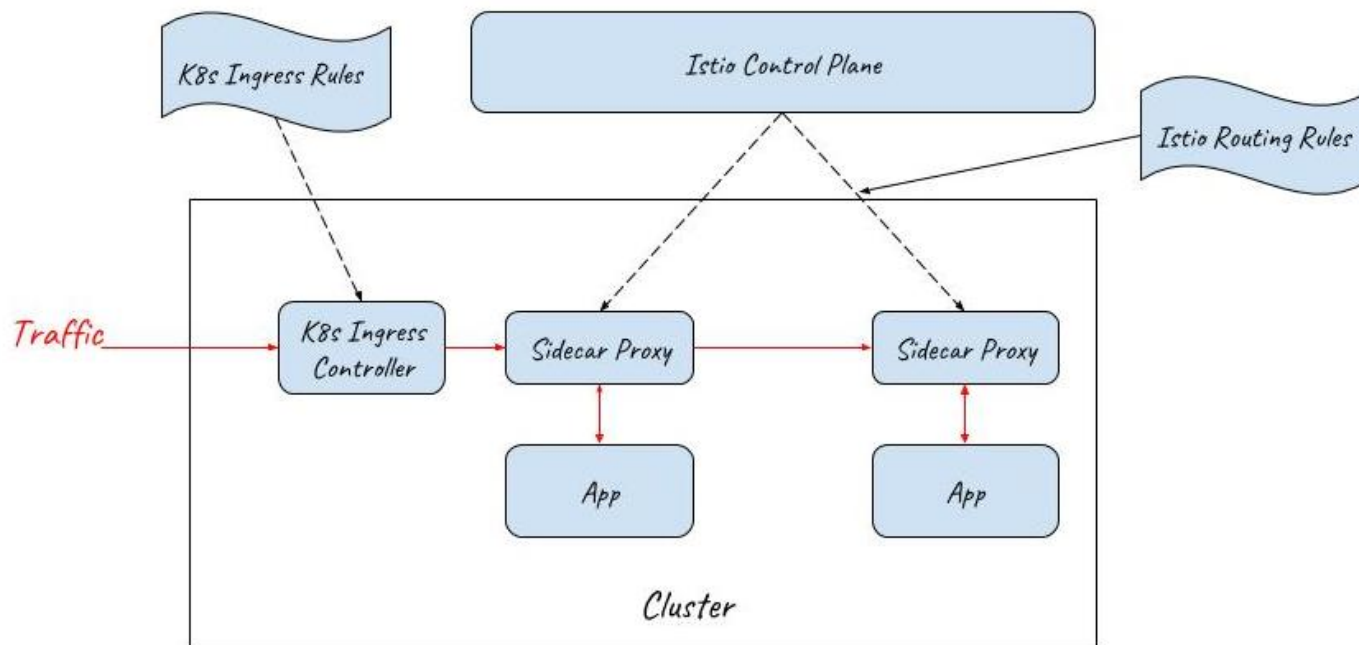
Key Settings

- ✓ Outlier Detection—The ability to automatically exclude abnormal (high error rate) instances from traffic
- ✓ To prevent increased user errors, delayed responses, and spread of failures

setting	Meaning
consecutive5xxErrors	Number of consecutive 5xx errors allowed
interval	inspection cycle
baseEjectionTime	Exclusion time
maxEjectionPercent	Maximum Exclusion Ratio

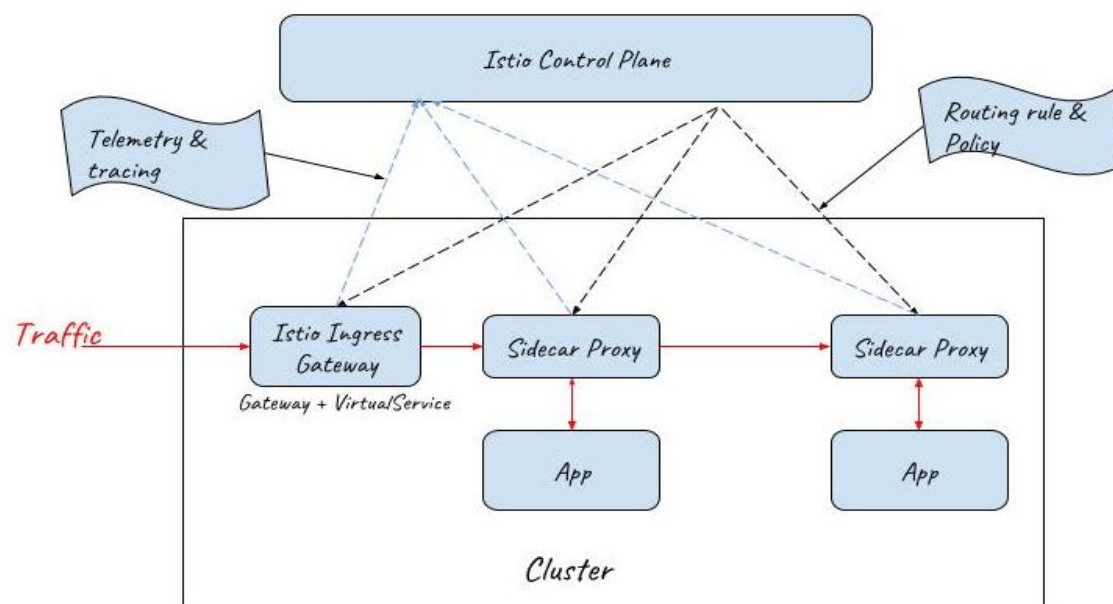
Ingress G/W in service mesh environment

- ✓ Ingress cannot be managed by Istio Control Plane
- ✓ Must be created with Kubernetes Ingress rule, does not fit well with Mesh
- ✓ Ingress provides only very basic L7 features
- ✓ Advanced routing rules, distributed tracking, policy checking, metrics collection, etc. are not provided with the same functionality as the Mesh Sidecar Proxy



Istio G/W

- ✓ Istio Gateway resources are much simpler than Kubernetes Ingress
- ✓ Only L4–L6 functions such as Port, Host, TLS key, and authentication can be configured.
- ✓ The gateway is bound to the Istio VirtualService resource, the same resource used for routing configuration within the mesh. This provides the same functions mentioned above inside and at the entrance of the mesh



Features that need to be added in Istio G/W



K8S Ingress

Load balancing
SSL termination
Virtual hosting



Istio Gateway

Load balancing
SSL termination
Virtual hosting
Advanced traffic routing
Fault injection
Other benefits brought
by Istio

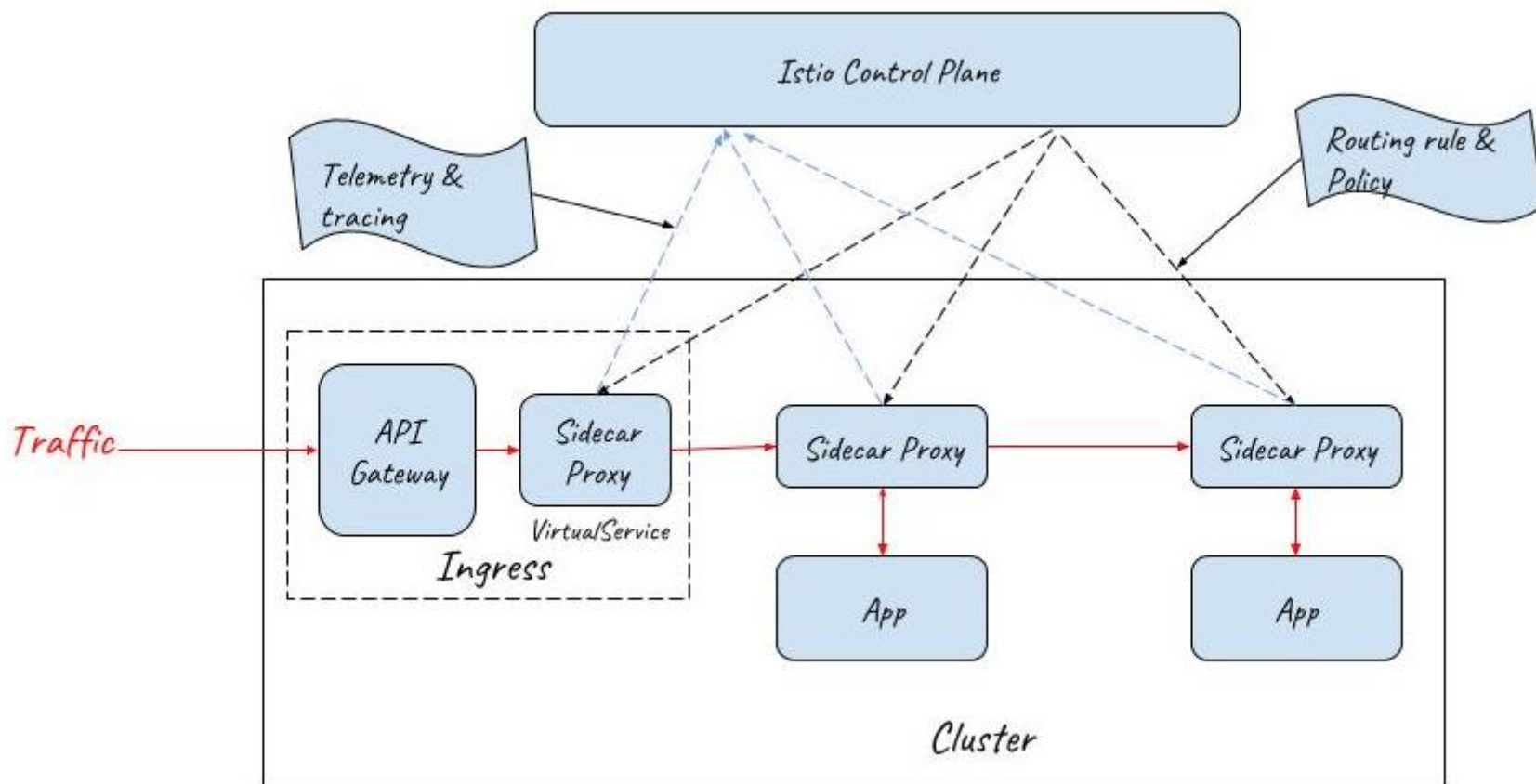


API Gateway

Load balancing
SSL termination
Virtual hosting
Traffic routing
API lifecycle management
API Operation Monitoring
API access authorization
API key management
API transformation
API Billing and Rate limiting

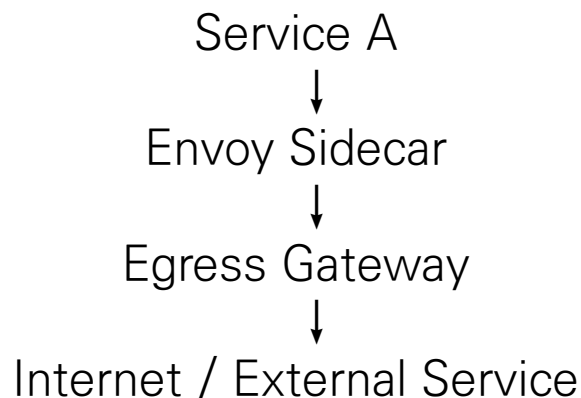
API Gateway + Sidecar Proxy

- ✓ API Gateway and Sidecar Proxy Use as Input Gateway for Service Mesh
- ✓ The API gateway already has L7 capabilities, so the Sidecar proxy only needs to provide routing capabilities for Istio VirtualService resources

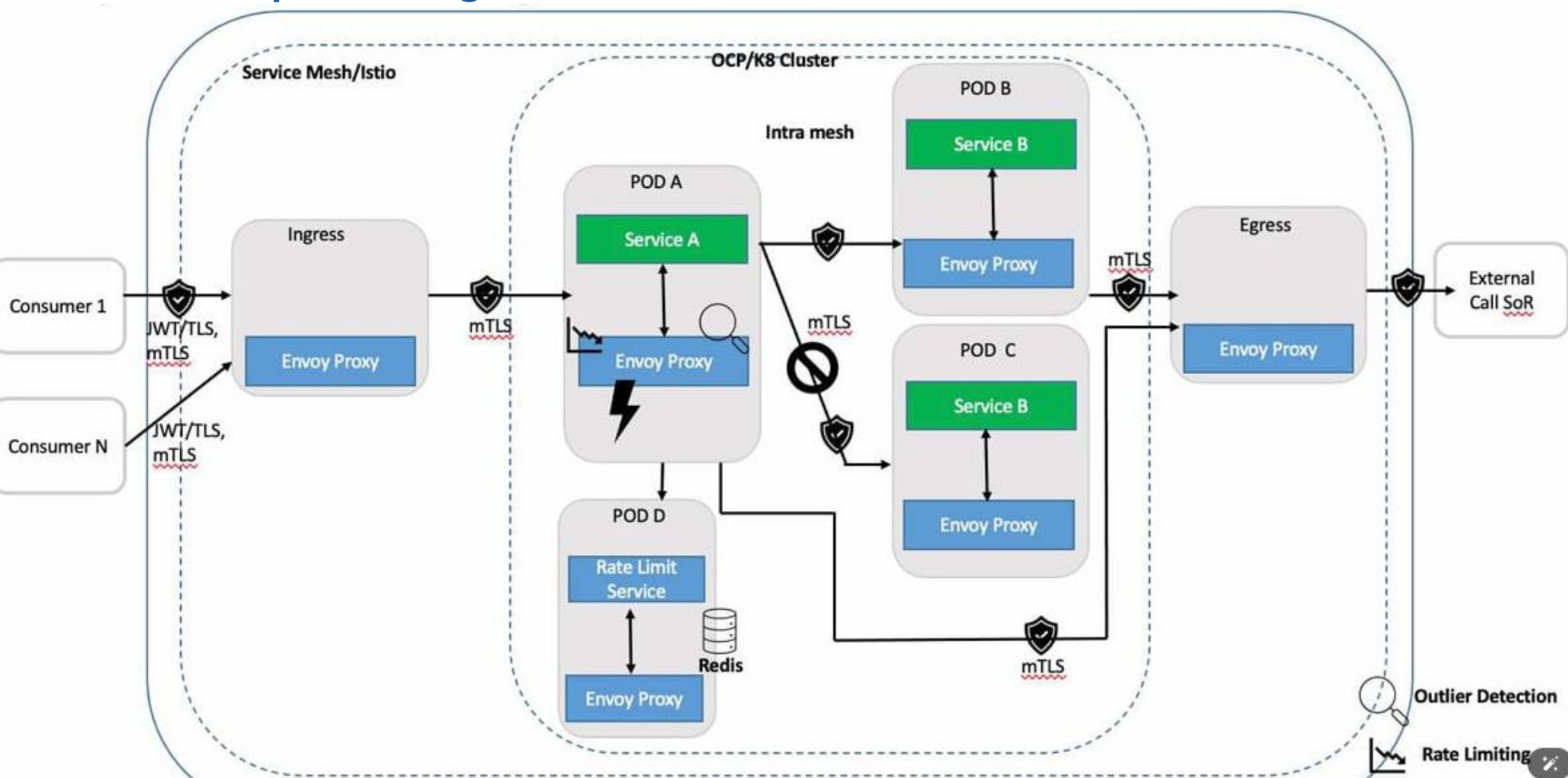


Purpose

- ✓ Egress controls traffic when the mesh internal service goes out of the external system (out of the cluster)
 - External Access Control
 - Access only allowed domains
 - TLS Policy Audit Logs
 - security, traffic monitoring
- ✓ Internal service calls below
 - Calling the Stripe API
 - External MySQL connection
 - Kafka SaaS
 - Google API
 - External REST API



a conceptual diagram



How to use it

- ✓ Use ServiceEntry to specify external access restrictions
 - DNS registration, TLS policy, Egress Gateway linkage, external service cataloging
 - `meshConfig.outboundTrafficPolicy.mode`
 - => ALLOW: Allow unregistered external addresses as well
 - => REGISTRY_ONLY: Allow ServiceEntry Registration Only

Function	Effect
Security	Prevent reckless external calls
inspection	Track all outbound
Policy	Allow only certain SaaS
NAT Control	Fixed Exit IP
Zero Trust	Enhance

- ✓ `istioctl uninstall --purge -y`
- ✓ `kubectl delete namespace istio-system`
- ✓ `kubectl delete namespace istio-ingress`
- ✓ `kubectl delete namespace istio-egress`

- ✓ `kubectl label namespace default istio-injection-`
- ✓ `kubectl rollout restart deploy`